

Manejo de errores con @ControllerAdvice

Java con Spring Boot de 0 a Pro • Video #7 • APIs REST con Spring Boot



¿Qué aprenderás hoy?

Objetivos de aprendizaje

- Comprender la importancia de manejar errores de forma uniforme en una API
- Dominar el uso de `@ControllerAdvice` y `@ExceptionHandler`
- Crear objetos de respuesta de error consistentes
- Personalizar mensajes y códigos HTTP según el contexto

Agenda del video

1. Problemas sin manejo centralizado
2. Introducción a `@ControllerAdvice`
3. Estructura del `ErrorDTO`
4. Ejemplo práctico con excepciones
5. Demo en vivo con Postman

 **Prerrequisitos:** Haber visto los videos anteriores sobre controladores, DTOs y validación con Bean Validation.

Por qué necesitas un manejo uniforme de errores

Sin un manejo centralizado de excepciones, tu API puede devolver respuestas inconsistentes, dificultando la integración para los clientes y aumentando el tiempo de depuración. Los usuarios finales experimentan mensajes confusos, y tu equipo pierde tiempo rastreando problemas que podrían prevenirse con una estrategia sólida.

Consistencia

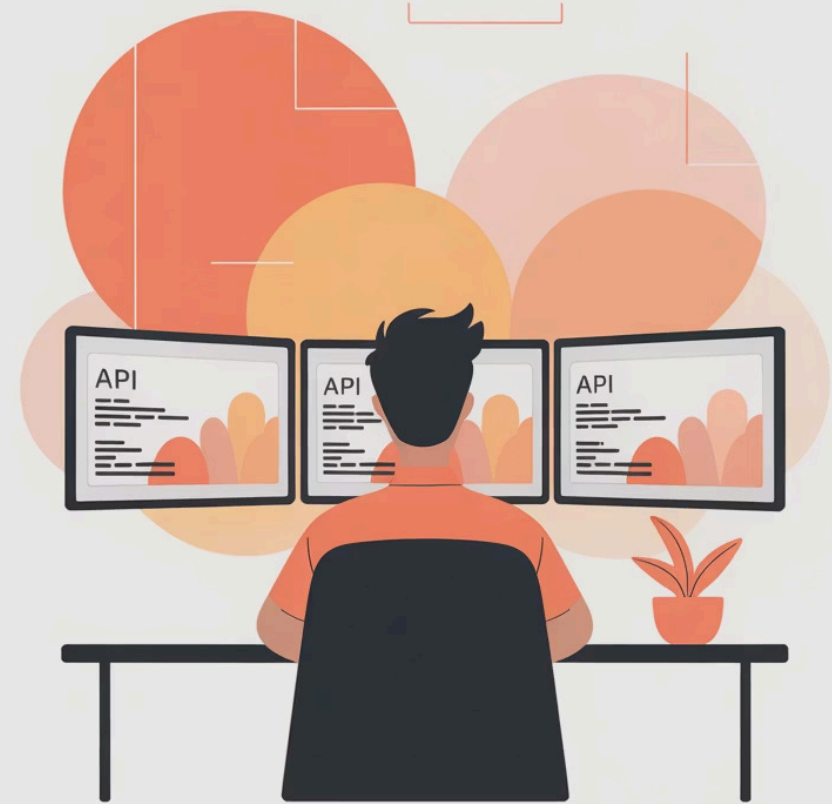
Todos los errores siguen el mismo formato JSON, facilitando el parseo automático en el frontend

Profesionalismo

Mensajes claros y códigos HTTP correctos demuestran calidad en tu API

Mantenibilidad

Centralizar la lógica de errores en un solo lugar simplifica futuras modificaciones





Problemas comunes sin manejo global

Stack traces expuestos

Los errores internos revelan detalles de implementación que no deberían ser públicos, comprometiendo la seguridad y profesionalismo de tu API.

Mensajes inconsistentes

Cada controlador maneja errores de forma diferente, generando respuestas JSON con estructuras variables que confunden a los consumidores de la API.

Códigos HTTP incorrectos

Devolver siempre 500 Internal Server Error cuando el problema es una validación (400) o recurso no encontrado (404) dificulta el diagnóstico correcto.

Duplicación de código

Repetir la lógica de manejo de errores en múltiples controladores aumenta el esfuerzo de mantenimiento y la probabilidad de bugs.

¿Qué es @ControllerAdvice?

@ControllerAdvice es una anotación de Spring que permite interceptar excepciones de forma global, aplicando lógica común a todos los controladores de tu aplicación. Funciona como un punto centralizado donde defines cómo responder ante diferentes tipos de errores.

@ExceptionHandler se combina con @ControllerAdvice para especificar qué excepciones capturar y cómo procesarlas, permitiendo respuestas personalizadas según el tipo de error.

01

Se lanza una excepción en cualquier controlador

02

@ControllerAdvice intercepta la excepción antes de llegar al cliente

03

@ExceptionHandler procesa el error según su tipo

04

Se devuelve una respuesta JSON estructurada con el código HTTP apropiado

Estructura del JSON de error: ErrorDTO

Un objeto ErrorDTO bien diseñado proporciona toda la información necesaria para diagnosticar y resolver problemas, manteniendo la consistencia en toda tu API.

☐ timestamp

Momento exacto en que ocurrió el error, útil para correlacionar con logs

☐ status

Código HTTP numérico (400, 404, 500, etc.)

☐ mensaje

Descripción legible del problema para el consumidor de la API

☐ detalles

Información adicional o lista de errores de validación específicos

Ejemplo de respuesta JSON

```
{
  "timestamp": "2024-01-15T10:30:45",
  "status": 400,
  "mensaje": "Error de validación",
  "detalles": [
    "El campo 'email' no es válido",
    "El campo 'nombre' es obligatorio"
  ]
}
```

Ejemplo práctico: IllegalArgumentException

Veamos cómo capturar y manejar una excepción específica, devolviendo una respuesta personalizada con el código HTTP correcto.



Código del manejador

```
@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<ErrorDTO> handleIllegalArgumentException(
    IllegalArgumentException ex) {

    ErrorDTO error = new ErrorDTO(
        LocalDateTime.now(),
        400,
        "Argumento inválido",
        ex.getMessage()
    );

    return ResponseEntity
        .badRequest()
        .body(error);
}
```

Respuesta en Postman

```
{
  "timestamp": "2024-01-15T11:45:30",
  "status": 400,
  "mensaje": "Argumento inválido",
  "detalles": "El ID debe ser mayor a 0"
}
```

El cliente recibe información clara y accionable en lugar de un error genérico.

Checklist para la demostración

1

Crear GlobalExceptionHandler

Clase anotada con `@ControllerAdvice` que centralizará todo el manejo de errores

2

Definir ErrorDTO

Record o clase con los campos: timestamp, status, mensaje y detalles

3

Implementar `@ExceptionHandler`

Métodos para `IllegalArgumentException` y `MethodArgumentNotValidException`

4

Simular errores en endpoints

Probar validaciones fallidas y búsquedas de recursos inexistentes

5

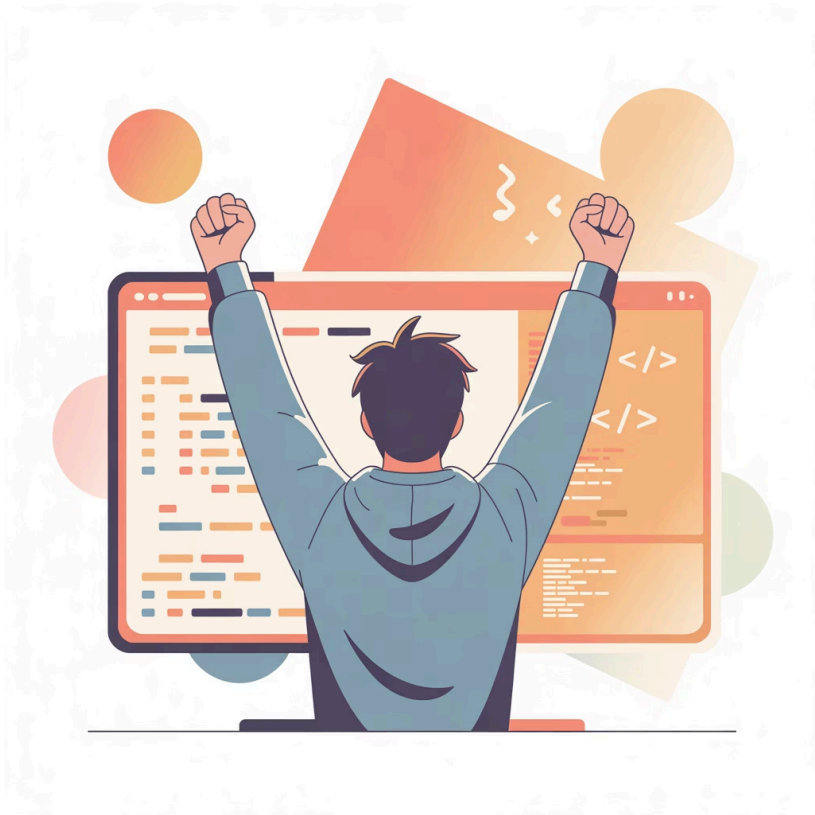
Verificar respuestas en Postman

Confirmar códigos HTTP correctos y formato JSON consistente

6

Revisar logs en consola

Asegurar que los errores se registran apropiadamente para debugging



Resumen: Buenas prácticas

- Centraliza el manejo de errores

Un solo `@ControllerAdvice` para toda la aplicación mantiene la consistencia y facilita el mantenimiento

- Usa códigos HTTP apropiados

400 para errores de validación, 404 para recursos no encontrados, 500 solo para errores del servidor

- Estructura consistente de respuesta

El mismo formato JSON facilita el consumo de tu API por diferentes clientes

- Mensajes claros y accionables

Ayuda a los desarrolladores a entender y resolver problemas rápidamente

- No expongas detalles internos

Protege información sensible de implementación en mensajes de error públicos



Próximos pasos en tu journey

Has dominado el manejo de errores con `@ControllerAdvice`, una habilidad fundamental para crear APIs REST profesionales. Ahora estás listo para explorar temas más avanzados que llevarán tu aplicación al siguiente nivel.



Seguridad con Spring Security

Protege tus endpoints con autenticación y autorización robustas



Paginación y filtrado

Maneja grandes volúmenes de datos eficientemente en tus respuestas



Documentación con OpenAPI

Genera documentación interactiva automáticamente para tu API



Testing de APIs REST

Aprende a escribir tests unitarios e integración para tus endpoints

¡Nos vemos en el próximo video de la serie! Java con Spring Boot de 0 a Pro